SQL Query Optimization: 15 Techniques for Better Performance

Explore different strategies for faster and more efficient SQL queries.

Jan 30, 2025 · 14 min read

Maria Eugenia Inzaugarat

Passionate Data Scientist always looking for challenges. Loves writing about Data Science and Python.

TOPICS

Data Engineering

SQL

SQL is an important tool for anyone managing and manipulating data within relational databases. It enables us to interact with databases and perform essential tasks efficiently. With the amount of available data growing every day, we face the challenge of writing complex queries to retrieve that data.

Slow queries can be a real bottleneck, impacting everything from application performance to user experience. Optimizing SQL queries improves performance, reduces resource consumption, and ensures scalability.

In this article, we'll look into some of the most effective techniques for optimizing our SQL queries. We will dive into the benefits and drawbacks of each technique to understand their impact on SQL query performance. Let's start!

1. Use Proper Indexing

Imagine we are searching for a book in a library without a catalog. We'd have to check every shelf and every row until we finally found it. Indexes in a database are similar to catalogs. They help us quickly locate the data we need without scanning the entire table.

How indexes work

Indexes are data structures that improve the speed at which data is retrieved. They work by creating a sorted copy of the indexed columns, which allows the database to quickly pinpoint the rows that match our query, saving us a lot of time.

There are three main types of indexes in databases:

- **Clustered indexes** - Physically order data based on column values and are best used for sequential or sorted data with no duplicates, such as primary keys.
- **Non-clustered indexes** - Create two separate columns, making them suitable for mapping tables or glossaries.
- **Full-text indexes** - Used to search large text fields, like articles or emails, by storing the positions of terms within the text.

So, how can we use indexes to improve the performance of SQL queries? Let's see some best practices:

- **Index frequently queried columns**. If we usually search a table using `customer_id` or `item_id`, indexing those columns will greatly impact speed. Check below how to

create an index:

```
CREATE INDEX index_customer_id ON customers (customer_id);
```

POWERED BY datalab

- Avoid using unnecessary indexes. While indexes are very helpful for speeding up `SELECT` queries, they can slightly slow down `INSERT`, `UPDATE`, and `DELETE` operations. This is because the index needs to be updated every time you modify data. So, too many indexes can slow things down by increasing the overhead for data modifications.
- **Choose the right index type.** Different databases offer various index types. We should pick the one that best suits our data and query patterns. For example, a B-tree index is a good choice if we often search for ranges of values.

2. Avoid SELECT *

Sometimes, we are tempted to use `SELECT *` to grab all columns, even those that aren't relevant to our analysis. Although this could seem convenient, it leads to very inefficient queries that can slow down performance.

The database has to read and transfer more data than necessary, requiring higher memory usage since the server must process and store more information than needed.

As a general best practice, we should only select the specific columns we need. Minimizing unnecessary data not only will keep our code clean and easy to understand but also help optimize the performance.

So, instead of writing:

```
SELECT *
FROM products;
```

POWERED BY datalab

We should write:

```
SELECT product_id, product_name, product_price
FROM products;
```

POWERED BY datalab

3. Avoid Redundant or Unnecessary Data Retrieval

We've just discussed that selecting only relevant columns is considered a best practice to optimize SQL queries. However, it is also important to limit the number of rows we are retrieving, not just columns. The queries usually slow down when the number of rows increases.

We can use `LIMIT` to reduce the number of rows returned. This feature prevents us from unintentionally retrieving thousands of rows of data when we only need to work with a few.

The `LIMIT` function is especially helpful for validation queries or inspecting the output of a transformation we're working on. It's ideal for experimentation and understanding how our code behaves. However, it may not be suitable for automated data models, where we need to return the entire dataset.

Here we have an example of how `LIMIT` works:

```
SELECT name
FROM customers
ORDER BY customer_group DESC
LIMIT 100;
```

POWERED BY datalab

4. Use Joins Efficiently

When working with relational databases, data is often organized into separate tables to avoid redundancy and improve efficiency. However, this means that we need to retrieve data from different places and glue them together to get all the relevant information we need.

Joins allow us to combine rows from two or more tables based on a related column between them in a single query, making it possible to perform more complex analyses.

There are different types of join, and we need to understand how to use them. Using the wrong join can create duplicates in our dataset and slow it down.

- An **inner join** returns only the rows with a match in both tables. If a record exists in one table but not the other, that record will be excluded from the result.

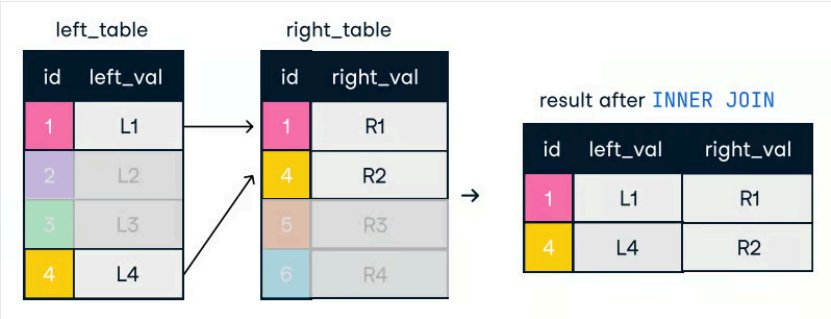
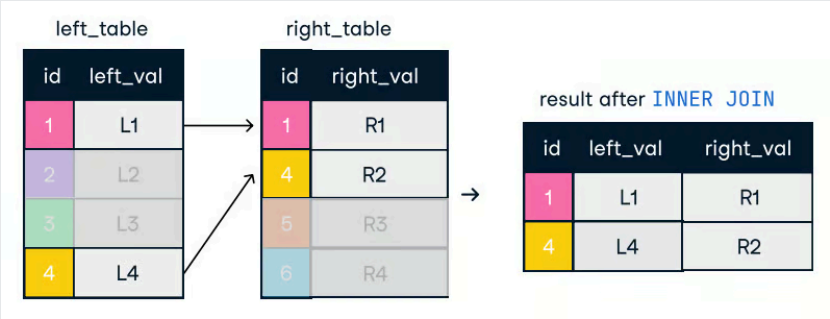


Figure: Inner Join. Image source: DataCamp SQL-Join cheat sheet.

```
SELECT o.order_id, c.name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

POWERED BY datalab

- An **outer join** returns all rows from one table and the matching rows from the other. If there is no match, NULL values are returned for columns from the table with no matching row.



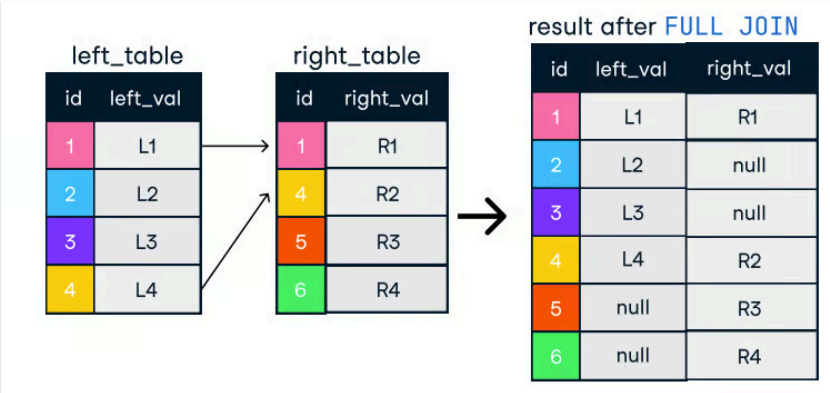


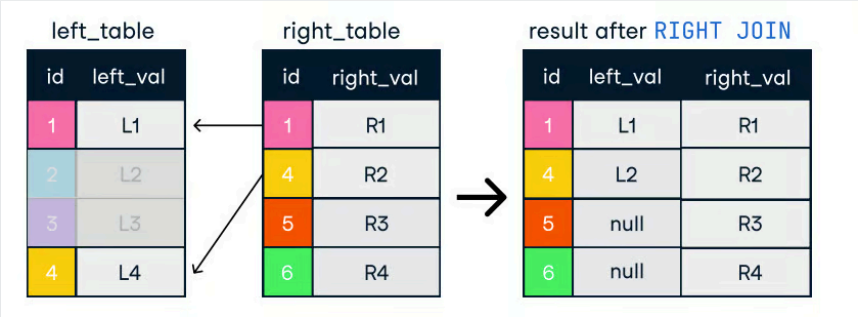
Figure: Outer or Full Join. Image source: DataCamp SQL-Join cheat sheet.

```
SELECT o.order_id, c.name
FROM orders o
```

```
FULL OUTER JOIN customers c ON o.customer_id = c.customer_id;
```

POWERED BY datalab

- A **left join** includes all rows from the left table and the matched rows from the right table. If no match is found, NULL values are returned for the right table's columns.
- Similarly, a **right join** includes all rows from the right table, with matching rows from the left, filling in NULLs where no match exists.



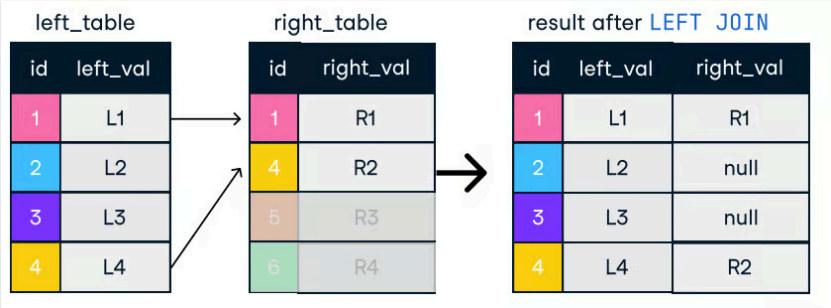


Figure: Left and Right Join. Image source: DataCamp SQL-Join cheat sheet.

```
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

POWERED BY datalab

Tips for efficient joins:

- **Order joins logically.** We should start with the tables that return the fewest rows. This reduces the amount of data that needs to be processed in subsequent joins.
- **Use indexes on join columns.** Again indexes are our ally. Using indexes helps the database quickly find matching rows.
- **Consider using subqueries or CTEs (Common Table Expressions)** to simplify complex joins:

```
WITH RecentOrders AS (
    SELECT customer_id, order_id
    FROM orders
    WHERE order_date >= DATE('now', '-30 days')
)
SELECT c.customer_name, ro.order_id
FROM customers c
INNER JOIN RecentOrders ro ON c.customer_id = ro.customer_id;
```

POWERED BY datalab

5. Analyze Query Execution Plans

Most of the time, we run SQL queries and only check if the output or result retrieved is what we expected. However, we rarely wonder what happens behind the scenes when we execute a SQL query.

Most databases provide functions such as `EXPLAIN` or `EXPLAIN PLAN` to visualize this process. These plans provide a step-by-step breakdown of how the database will retrieve the data. We can use this feature to identify where we have the performance bottlenecks and make informed decisions about optimizing our queries.

Let's see how we can use `EXPLAIN` to identify bottlenecks. We'll run the following code:

```
EXPLAIN SELECT f.title, a.actor_name
FROM film f, film_actor fa,  actor a
WHERE f.film_id = fa.film_id and fa.actor_id = a.id
```

POWERED BY datalab

We can then examine the results:

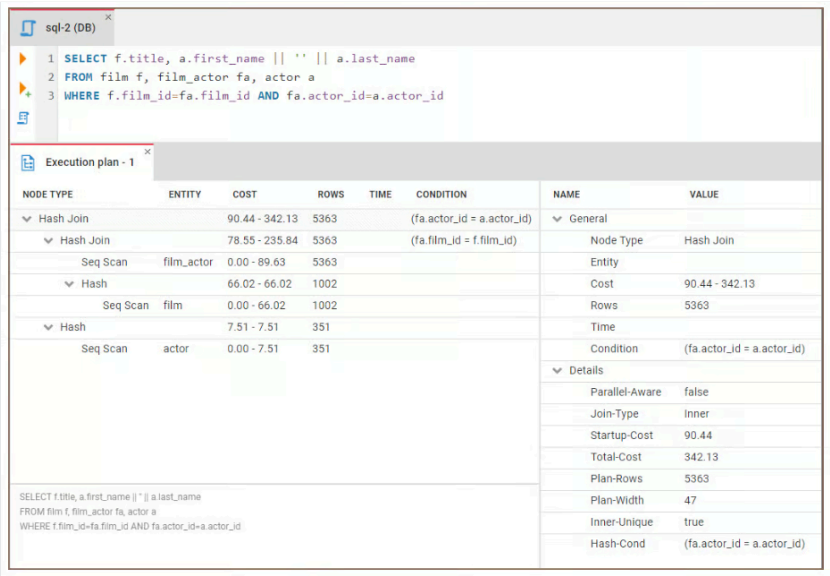


Figure: An example of a query execution plan. Image source: CloudDBeaver website.

Here's general guidance on how to interpret the results:

- **Full table scan:** If the plan shows a full table scan, the database scans every row in the table, which can be painfully slow. This often indicates a missing index or an inefficient `WHERE` clause.
- **Inefficient join strategies:** The plan can reveal if the database uses a less optimal join algorithm.
- **Other potential issues:** Explain plans can highlight other problems, such as high sort costs or excessive temporary table usage.

Associate Data Engineer in SQL
Gain practical knowledge in ETL, SQL, and data warehousing for data engineering.

Explore Track

6. Optimize WHERE Clauses

The `WHERE` clause is essential in SQL queries because it allows us to filter data based on specific conditions, ensuring only relevant records are returned. It improves query efficiency by reducing the amount of data processed, which is very important for working with a large dataset.

So, a correct `WHERE` clause can be a powerful ally when we are optimizing the performance of a SQL query. Let's see some ways in which we can take advantage of this

clause:

- **Add proper filtering conditions early.** Sometimes, having a `WHERE` clause is good but not enough. We have to be careful where we place the clause. Filtering out as many rows as possible early in the `WHERE` clause can help us optimize the query.
- **Avoid using functions on columns in the `WHERE` clause.** When we apply a function to a column, the database has to apply that function to every row in the table before it can filter the results. This prevents the database from using indexes effectively.

For example, instead of:

```
SELECT *
FROM employees WHERE
YEAR(hire_date) = 2020;
```

POWERED BY datalab

We should use:

```
SELECT *
FROM employees
WHERE hire_date >= '2020-01-01' AND hire_date < '2021-01-01';
```

POWERED BY datalab

- **Use appropriate operators.** We should choose the most efficient operators that meet our needs. For example, `=` is generally faster than `LIKE`, and using specific date ranges is faster than using functions like `MONTH(order_date)`.

So, for example, instead of performing this query:

```
SELECT *
FROM orders
WHERE MONTH(order_date) = 12 AND YEAR(order_date) = 2023;
```

POWERED BY datalab

We can perform the following:

```
SELECT *
FROM orders
WHERE order_date >= '2023-12-01' AND order_date < '2024-01-01';
```

POWERED BY datalab

7. Optimize Subqueries

In some cases, we are writing a query and feel we need to dynamically perform some filtering, aggregation, or joins of data. We don't want to do several queries; instead, we want to keep it to a single query.

For those cases, we can use **subqueries**. Subqueries in SQL are queries nested inside another query, typically in the `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statements.

Subqueries can be powerful and fast, but they can also cause performance issues if they are not used carefully. As a rule, we should minimize the use of subqueries and follow a set of best practices:

- **Replace subqueries with joins when possible.** Joins are generally faster and more efficient than subqueries.
- **Use common table expressions (CTEs) instead.** **CTEs** separate our code into a few smaller rather than one big query, which is much easier to read.

```
WITH SalesCTE AS (
             SELECT salesperson_id, SUM(sales_amount) AS total_sales
             FROM sales GROUP BY salesperson_id )

SELECT salesperson_id, total_sales
FROM SalesCTE WHERE total_sales > 5000;
```

POWERED BY datalab

- **Use uncorrelated subqueries.** Uncorrelated subqueries are independent of the outer query and can be executed once, whereas correlated subqueries are executed for each row in the outer query.

8. Use EXISTS Instead of IN for Subqueries

When working with subqueries, we often need to check if a value exists in a set of results. We can do this with two `IN` or `EXISTS`, but `EXISTS` is generally more efficient, especially for larger datasets.

The `IN` clause reads the entire subquery result set into memory before comparing. On the other hand, the `EXISTS` clause stops processing the subquery as soon as it finds a match.

Here we have an example of how to use this clause:

```
SELECT *
FROM orders o
WHERE EXISTS (SELECT 1 FROM customers c WHERE c.customer_id = o.customer_id AND c
```

POWERED BY datalab

9. Limit the use of DISTINCT

Imagine that we are working on an analysis for sending a promotional offer to customers from unique cities. The database has multiple orders from the same customers. The first thing that pops into our mind is using the `DISTINCT` clause.

This function is handy for certain cases but can be resource-intensive, especially on large datasets. There are a few alternatives to `DISTINCT`:

- **Identify and remove duplicate data during data cleaning processes.** This prevents duplicates from creeping into our database in the first place.
- **Use `GROUP BY` instead of `DISTINCT` when possible.** `GROUP BY` can be more efficient, especially when combined with aggregate functions.

So, instead of performing:

```
SELECT DISTINCT city FROM customers;
```

POWERED BY datalab

We can use:

```
SELECT city FROM customers GROUP BY city;
```

POWERED BY datalab

- Use window functions. Window functions such as `ROW_NUMBER` can help us identify duplicates and filter them out without using `DISTINCT`.

10. Leverage Database-Specific Features

When working with data, we interact with it using SQL through a Database Management System (DBMS). The DBMS processes the SQL commands, manages the database, and

ensures data integrity and security. Different database systems offer unique features that can help optimize queries.

Database hints are special instructions we can add to our queries to execute a query more efficiently. They are a helpful tool, but they should be used with caution.

For example, in **MySQL**, the `USE INDEX` hint can force the use of a specific index:

```
SELECT * FROM employees USE INDEX (idx_salary) WHERE salary > 50000;
```

POWERED BY datalab

In SQL Server, the `OPTION (LOOP JOIN)` hint specifies the join method:

```
SELECT *
FROM orders
INNER JOIN customers ON orders.customer_id = customers.id OPTION (LOOP JOIN);
```

POWERED BY datalab

These hints override the default query optimization, improving performance in specific scenarios.

On the other hand, partitioning and sharding are two techniques for distributing data in the cloud.

- With partitioning, we divide one large table into multiple smaller tables, each with its partition key. Partition keys are typically based on the timestamps of when rows were created or the integer values they contain. When we execute a query on this table, the server will automatically route us to the partitioned table appropriate for our query.
- Sharding is quite similar, except that instead of splitting one big table into smaller tables, it's splitting one big database into smaller databases. Each of these databases is on a different server. Instead of a partition key, a sharding key redirects queries to be run on the appropriate database. Sharding increases processing speeds because the load is split across different servers.

11. Monitor and Optimize Database Statistics

Keeping database statistics up to date is important to ensure that the query optimizer can make informed, accurate decisions regarding the most efficient way to execute queries.

Statistics describe the data distribution in a table (e.g., the number of rows, the frequency of values, and the spread of values across columns), and the optimizer relies on this information to estimate query execution costs. If statistics are outdated, the optimizer may choose inefficient execution plans, such as using the wrong indexes or opting for a full table scan instead of a more efficient index scan, leading to poor query performance.

Databases often support automatic updates to maintain accurate statistics. For instance, in SQL Server, the default configuration automatically updates statistics when a significant amount of data changes. Similarly, PostgreSQL has an auto-analyze feature, which updates statistics after a specified threshold of data modification.

However, we can manually update statistics in cases where automatic updates are insufficient or if manual intervention is needed. In SQL Server, we can use the `UPDATE STATISTICS` command to refresh the statistics for a specific table or index, while in PostgreSQL, the `ANALYZE` command can be run to update statistics for one or more tables.

```
-- Update statistics for all tables in the current database
ANALYZE;

-- Update statistics for a specific table
ANALYZE my_table;
```

12. Utilize stored procedures

A stored procedure is a set of SQL commands we save in our database so we don't have to write the same SQL repeatedly. We can think of it as a reusable script.

When we need to perform a certain task, like updating records or calculating values, we just call the stored procedure. It can take input, do some work, such as querying or modifying data, and even return a result. Stored procedures help speed things up since the SQL is precompiled, making your code cleaner and easier to manage.

We can create a stored procedure in PostgreSQL as follows:

```
CREATE OR REPLACE PROCEDURE insert_employee(
    emp_id INT,
    emp_first_name VARCHAR,
    emp_last_name VARCHAR
)
LANGUAGE plpgsql
AS $
BEGIN
    -- Insert a new employee into the employees table
    INSERT INTO employees (employee_id, first_name, last_name)
    VALUES (emp_id, emp_first_name, emp_last_name);
END;
$;

-- call the procedure
CALL insert_employee(101, 'John', 'Doe');
```

POWERED BY datalab

13. Avoid Unnecessary Ordering and Grouping

As data practitioners, we like to have our data ordered and grouped so we can gain insights more easily. We usually use `ORDER BY` and `GROUP BY` in our SQL queries.

However, both clauses can be computationally expensive, especially when dealing with large datasets. When sorting or aggregating data, the database engine must often perform a full scan of the data and then organize it, identify the groups, and/or apply aggregate functions, typically using resource-intensive algorithms.

To optimize the queries, we can follow some of these tips:

- **Minimize sorting**. We should only use `ORDER BY` when necessary. If sorting is not essential, omitting this clause can help us dramatically reduce processing time.
- **Use indexes.** When possible, we should make sure that the columns involved in `ORDER BY` and `GROUP BY` are indexed.
- **Push sorting to the application layer**. If it is possible, we should push the sorting operation to the application layer rather than the database.
- **Pre-aggregate data**. For complex queries involving `GROUP BY`, we could pre-aggregate the data at an earlier stage or in a **materialized view**, so the database doesn't need to compute the same aggregates repeatedly.

14. Use UNION ALL Instead of UNION

When we want to combine results from multiple queries into one list, we can use the `UNION` and `UNION ALL` clauses. Both combine the results of two or more `SELECT` statements when they have the same column names. However, they are not the same, and their difference makes them suitable for different use cases.

The `UNION` clause removes duplicate rows, which requires more processing time.

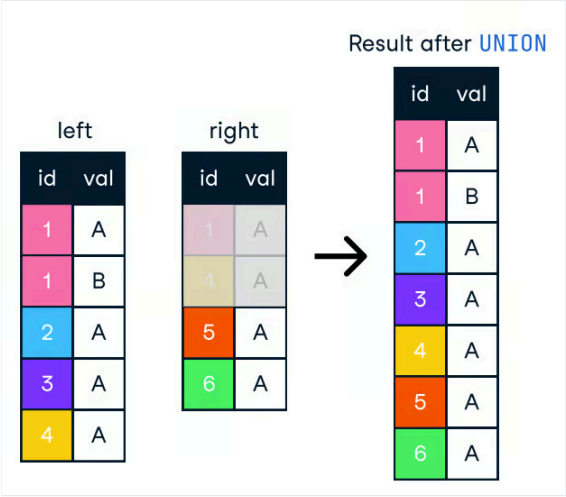


Figure: Union in SQL. Image source: DataCamp SQL-Join cheat sheet.

On the other hand, `UNION ALL` combines the results but keeps all rows, including duplicates. So, if we don't need to remove duplicates, we should use `UNION ALL` for better performance.

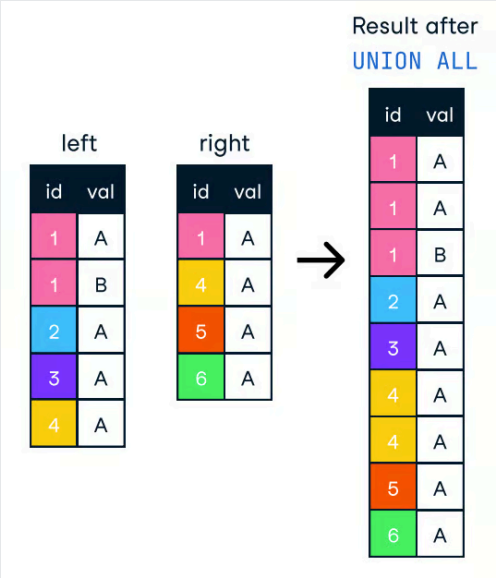


Figure: UNION ALL in SQL. Image source: DataCamp SQL-Join cheat sheet.

```sql
-- Potentially slower
SELECT product_id FROM products WHERE category = 'Electronics'
UNION
SELECT product_id FROM products WHERE category = 'Books';

-- Potentially faster
SELECT product_id FROM products WHERE category = 'Electronics'
UNION ALL
SELECT product_id FROM products WHERE category = 'Books';
```

POWERED BY datalab

15. Break Down Complex Queries

Working with large datasets implies that we'll often encounter complex queries that are difficult to understand and optimize. We can try to tackle these cases by breaking them down into smaller, simpler queries. In this way, we can easily identify performance bottlenecks and apply optimization techniques.

One of the most frequently used strategies to break down queries is materialized views. These are precomputed and stored query results that can be accessed quickly rather than recalculating the query each time it's referenced. When the underlying data changes, the materialized view must be manually or automatically refreshed.

Here's an example of how to create and query a materialized view:

```sql
-- Create a materialized view
CREATE MATERIALIZED VIEW daily_sales AS
SELECT product_id, SUM(quantity) AS total_quantity
FROM order_items
GROUP BY product_id;

-- Query the materialized view
SELECT * FROM daily_sales;
```

POWERED BY datalab

Conclusion

In this article, we have explored various strategies and best practices for optimizing SQL queries, from indexing and joins to subqueries and database-specific features. By applying these techniques, you can significantly improve the performance of your queries and make our databases run more efficiently.

Remember, optimizing SQL queries is an ongoing process. As your data grows and your application evolves, you'll need to continually monitor and optimize your queries to ensure they're running at optimal performance.

To further enhance your understanding of SQL, we encourage you to explore the following resources on DataCamp:

- SQL Basics Cheat Sheet
- Improving Query Performance in SQL Server
- Improving Query Performance in PostgreSQL

Become SQL Certified
Prove your SQL skills are job-ready with a certification.

Boost My Career

AUTHOR
Maria Eugenia Inzaugarat

TOPICS

Data Engineering SQL

Training more people?
Get your team access to the full DataCamp for business platform.

For Business

For a bespoke solution book a demo.

Learn more about SQL with these courses!

COURSE

Data Manipulation in SQL

4 hr 317.3K

Master the complex SQL queries necessary to answer a wide variety of data science questions and prepare robust data sets for analysis in PostgreSQL.

See Details →

Start Course

See More →

Related

TUTORIAL

SQL Tutorial: How To Write Better Queries

TUTORIAL

Dynamic SQL: Techniques, Security, and Optimization

TUTORIAL

SQL Order of Execution: Understanding How Queries Run

See More →

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.

Download on the App Store

GET IT ON Google Play

Python Courses

R Courses

SQL Courses

Power BI Courses

Tableau Courses

Alteryx Courses

Azure Courses

AWS Courses

Google Cloud Courses

Google Sheets Courses

Excel Courses

AI Courses

Data Analysis Courses

Data Visualization Courses

Machine Learning Courses

Data Engineering Courses

Probability & Statistics Courses

DATALAB

Get Started

Pricing

Security

Documentation

CERTIFICATION

Certifications

Data Scientist

Data Analyst

Data Engineer

SQL Associate

Power BI Data Analyst

Tableau Certified Data Analyst

Azure Fundamentals

AI Fundamentals

RESOURCES

Resource Center

Upcoming Events

Blog

Code-Alongs

Tutorials

Docs

Open Source

RDocumentation

Book a Demo with DataCamp for Business

Data Portfolio

PLANS

Pricing

For Students

For Business

For Universities

Discounts, Promos & Sales

Expense DataCamp

DataCamp Donates

FOR BUSINESS

Business Pricing

Teams Plan

Data & AI Unlimited Plan

Customer Stories

Partner Program

ABOUT

About Us

Learner Stories

Careers

Become an Instructor

Press

Leadership

Contact Us

DataCamp Español

DataCamp Português

DataCamp Deutsch

DataCamp Français

SUPPORT

Help Center

Become an Affiliate

clear as data

Privacy Policy Cookie Notice Do Not Sell My Personal Information Accessibility Security Terms of Use

© 2026 DataCamp, Inc. All Rights Reserved.